

1장 JSX 문법과 React 기본 개념

React 19의 핵심은 선언적 UI 작성 방식이며, 이를 가능하게 하는 JSX는 HTML과 JavaScript가 결합된 직관적 문법으로 컴포넌트 기반 화면을 정의하는 기초가 되므로, 본 장에서는 JSX의 정의와 작성 규칙을 시작으로 React DOM이 가상 DOM을 통해 실제 화면에 반영되는 구조, 단일 루트 노드 원칙의 필요성과 적용 방식, 그리고 React 프로젝트를 시작하기 위한 개발환경 구성 및 초기 프로젝트 생성 과정을 실습 중심으로 학습하여 이후 단계의 컴포넌트 설계와 상태 관리로 이어질 수 있는 탄탄한 기초를 다진다.

1.1 JSX 정의 및 작성 규칙

JSX는 HTML과 유사한 문법으로 JavaScript 코드 안에서 UI를 선언적으로 표현할 수 있게 해주며, 작성 규칙과 변환 과정을 이해하면 직관적이고 오류 없는 컴포넌트 코드를 작성할 수 있는 기초를 마련할 수 있다.

(1) JSX의 탄생 배경

JSX(JavaScript XML)는 **리액트(React)** 개발 환경에서 **사용자 인터페이스(UI)**를 구축하기 위해 탄생한 특별한 문법입니다. 언뜻 보면 HTML과 비슷하지만, 실제로는 자바스크립트의 확장 문법입니다. JSX가 탄생한 배경에는 당시 웹 개발의 복잡성과 한계를 극복하려는 페이스북(현 메타) 개발팀의 고민이 담겨 있습니다.

이에 UI와 로직을 분리하지 않고 하나의 컴포넌트 안에서 선언적으로 작성하기 위해 JSX가 도입되었으며, 이는 복잡한 UI를 직관적이고 유지보수하기 쉽게 만듭니다.

① 기존 방식의 문제점

리액트가 등장하기 전에는 웹 애플리케이션의 UI를 만들 때 **HTML, CSS, JavaScript**를 분리하여 작성하는 것이 일반적이었습니다. 이 방식은 각 기술의 역할을 명확히 구분한다는 장점이 있었지만, 다음과 같은 문제점들을 야기했습니다.

- **복잡한 동적 UI 관리의 어려움**

단순한 웹 페이지에서는 문제가 없었지만, **데이터에 따라 UI가 동적으로 변화하는 복잡한 애플리케이션**을 만들 때 어려움이 커졌습니다. 예를 들어, 서버에서 받은 데이터를 기반으로 새로운 `<div>` 요소를 수십 개 만들어야 하는 경우, JavaScript 코드로 일일이 `document.createElement('div')` 를 호출하고 속성을 추가하는 방식은 **코드의 가독성을 심하게 떨어뜨리고 유지보수를 어렵게** 만들었습니다.

- **마크업과 로직의 분리 문제:**

"관심사의 분리(Separation of Concerns)"라는 원칙에 따라 HTML(마크업), CSS(스타일), JavaScript(로직)를 분리하는 것이 이상적으로 여겨졌습니다. 그러나 **컴포넌트 기반**의 현대 웹 개발에서는 UI를 구성하는 마크업과 그 마크업을 제어하는 로직이 서로 떼어낼 수 없는 관계를 가집니다. 특정 UI 요소의 클릭 이벤트 핸들러나 데이터 바인딩 로직을 다른 파일에서 관리하는 방식은 오히려 **하나의 컴포넌트 전체를 이해하고 수정하기 어렵게** 만들었습니다.

② JSX의 해결책

이러한 문제점을 해결하기 위해 페이스북 개발팀은 **컴포넌트의 렌더링 로직(UI를 그리는 방법)과 마크업을 함께 관리**하는 방식을 제안했습니다. JSX는 이러한 아이디어를 구현한 결과물입니다.

JSX는 개발자가 익숙한 HTML과 유사한 문법으로 UI 구조를 직관적으로 작성할 수 있게 합니다. 이 코드는 **빌드 과정에서 일반 JavaScript 코드로 변환(Transpiling)**됩니다. 예를 들어, `<MyComponent />` 는 `React.createElement(MyComponent)` 와 같은 함수 호출로 변환되어 실행됩니다.

③ JSX의 장점

- **직관적이고 선언적인 UI 작성**

데이터가 어떻게 보일지를 **명령하는 방식(Imperative)** 대신, 데이터가 주어졌을 때 UI가 **어떻게 생겼으면 좋겠다**는 것을 선언적으로(Declarative) 표현할 수 있게 합니다. 이는 UI 코드를 훨씬 직관적으로 만들고, 복잡한 UI 구조를 쉽게 파악할 수 있도록 돕습니다.

- **마크업과 로직의 결합**

컴포넌트의 ****렌더링 로직(JavaScript)****과 ****마크업(UI)****을 하나의 파일에 함께 작성함으로써, 해당 컴포넌트의 모든 기능을 한눈에 파악할 수 있게 됩니다. 이는 코드의 응집도를 높이고 유지보수를 용이하게 만듭니다.

- **향상된 가독성과 생산성**

HTML과 유사한 문법 덕분에 개발자는 별도의 학습 없이도 빠르게 리액트 개발에 적응할 수 있습니다. 또한, 기존의 복잡한 `createElement` 호출 방식보다 훨씬 간결하게 코드를 작성할 수 있어 생산성이 크게 향상됩니다.

결론적으로, JSX는 복잡한 웹 애플리케이션의 UI를 효율적으로 구축하기 위해 **마크업과 로직을 결합하고, 직관적인 선언적 문법을 제공**하는 혁신적인 해결책으로 탄생했습니다. 이는 리액트가 현대 웹 개발에서 가장 널리 사용되는 라이브러리 중 하나가 되는 데 결정적인 역할을 했습니다.

(2) JSX의 기본 문법 구조

JSX는 HTML 태그 형태와 중괄호 `{ }` 내부의 JavaScript 표현식을 함께 사용할 수 있어, 정적 구조와 동적 데이터를 유연하게 결합해 UI를 정의한다.

JSX는 HTML 유사 문법 + `{ }` 안에 JavaScript 표현식을 넣는 방식입니다.

```
function Greeting({ name, items }) {  
  return (  
    <section>  
      <h1>Hello, {name}!</h1>  
      <ul>  
        {items.map(item => <li key={item.id}>{item.text}</li>)}  
      </ul>  
    </section>  
  );  
}
```

- `{ }` 내부에는 **expression(표현식)** 만 올 수 있습니다 — 문(statement)은 안 됩니다.
(예: `if` 대신 삼항 연산자 또는 별도 함수 사용)
- JSX는 결국 함수 호출 형태의 객체를 반환합니다(하단 Babel 변환 참고).
- 문자열 결합: `{ { Hello ${name} } }`은 불가능 — 템플릿 리터럴로 변수 결합 후 넣거나 JSX 내부에 여러 자식으로 나열.

(3) HTML과 다른 점

JSX는 브라우저 표준 HTML과 다르게 `class` 속성을 `className`으로, `for` 속성을 `htmlFor`로 작성해야 하며, 이는 JavaScript 예약어 충돌을 방지하기 위한 규칙이다.

JSX는 HTML과 매우 유사하지만, 몇몇 속성은 JavaScript 예약어나 DOM property와 매핑되도록 이름이 바뀝니다.

- `class` → `className` (JS에서 `class` 는 예약어/키워드)
- `for` → `htmlFor` (`for` 는 JS 반복문 예약어)
- `tabindex` → `tabIndex`, `readonly` → `readOnly` 등: 카멜케이스로 표기(대부분 DOM property 이름을 따름)

- 커스텀 data/aria 속성은 그대로: `data-id="x"`, `aria-label="..."`

```
<label htmlFor="email">Email</label>
<input id="email" className="input" />
```

주의: 잘못된 속성명(`class`, `for`) 사용 시 빌드 오류는 아닐 수 있으나 의도대로 동작하지 않습니다. ESLint 규칙으로 검출 가능.

(4) JSX 내부에서 JavaScript 표현식 활용

JSX 내부에서는 변수, 함수 호출, 삼항 연산자, 배열의 `map()` 을 통한 반복 처리 등 JavaScript 표현식을 자유롭게 사용하여 동적 UI를 효과적으로 구성할 수 있다.

JSX의 `{ }` 안에는 표현식을 넣어 동적 UI를 생성합니다.

- 리스트 렌더링: `array.map()` → 반드시 `key` prop 제공
- 조건부 렌더링: 삼항 연산자 `condition ? <A /> : <B />` 또는 `condition && <A />`
- 표현식 예: 함수 호출, 연산, 템플릿 리터럴 결과 등

```
{users.length ? (
  users.map(u => <UserCard key={u.id} user={u} />)
) : (
  <p>No users</p>
)}
```

- `key` 는 렌더링 성능과 안정성(재정렬 시 컴포넌트 재사용 판단)에 중요 — 고유하고 예측 가능한 값 사용(인덱스는 재정렬·삭제가 있는 리스트에선 피함).
- `&&` 사용 시 왼쪽 값이 `0`, `"` 같은 falsy라도 화면에 표시될 수 있음(`0 && <A />` → `0`). 숫자 0을 조건으로 할 수 있는 경우 삼항 사용 권장.
- 중괄호 안에서 `for` 문을 사용할 수 없음. 반복 로직은 `map` 처럼 표현식 기반으로 작성.

(5) 태그 닫기 규칙과 self-closing 태그 사용

JSX는 XML 문법을 기반으로 하기 때문에 모든 태그를 반드시 닫아야 하며, 내용이 없는 태그는 `<img />`, `<input />` 처럼 self-closing 형태로 작성해야 한다.

JSX는 XML/HTML과 달리 **모든 태그를 닫아야** 합니다(닫지 않으면 파싱 에러).

- 빈 태그는 self-closing: `` , `<input />` , `<br />`
- React 컴포넌트도 동일: `<MyComponent />` (닫는 태그 또는 self-closing)
- 멀티라인 문자열/주석: `{/* 주석 */}`

```
// 잘못된 예 (에러)
return ;

// 올바른 예
return ;
```

주의: HTML의 일부 허용 관습(예: `<input>` 닫음 생략)이 JSX에서는 통하지 않습니다.

(6) Babel에 의한 JSX → JavaScript 변환 과정

브라우저가 JSX를 직접 해석하지 못하기 때문에, Babel이 JSX를 `React.createElement()` 형태의 순수 JavaScript 코드로 변환하여 실행 가능하게 한다.

JSX

```
const el = <h1 className="title">Hello, {name}</h1>;
```

클래식(기존) 변환

```
const el = React.createElement("h1", { className: "title" }, "Hello, ", name);
```

자동 런타임(React 17+) 변환 (간소화된 형태):

```
import { jsx as _jsx } from "react/jsx-runtime";
const el = _jsx("h1", { className: "title", children: ["Hello, ", name] });
```

Babel 설정 (예: `.babelrc`)

```
{
  "presets": [
    ["@babel/preset-react", { "runtime": "automatic" }]
  ]
}
```

- `runtime: "automatic"` 면 `import React from 'react'` 없이도 JSX 사용 가능(React 17+ 권장).
- `runtime: "classic"` 이면 종전처럼 `React.createElement` 를 사용하므로 `import React from 'react'` 필요.

TypeScript의 경우 `.tsx` 확장자 필요하고, `tsconfig.json` 및 `@babel/preset-typescript` 설정을 병행.

(7) JSX 코드 작성 시 흔히 발생하는 오류와 해결 방법

여러 형제 태그 반환, 따옴표와 중괄호 혼용, 닫지 않은 태그 등으로 오류가 발생하기 쉬우며, ESLint 규칙과 오류 메시지를 통해 즉시 수정할 수 있다.

아래는 실무에서 자주 보는 오류와 원인·해결책입니다.

- **"Adjacent JSX elements must be wrapped"**
 - 원인: 컴포넌트가 여러 루트 엘리먼트를 반환.
 - 해결: `<div>` 나 `<React.Fragment>` (`<>...</>`)로 감싸기.

```
// 에러
return <h1/> <p/>;

// 해결
return (<><h1/><p/></></>);
```

- **"Unexpected token" (JSX 구문 오류)**
 - 원인: Babel/TSX 변환 설정 누락 또는 파일 확장자(.jsx/.tsx) 문제.
 - 해결: 빌드 설정 확인(`@babel/preset-react`, Vite/CRA 설정), 올바른 확장자 사용.
- **`class` 나 `for` 사용으로 의도 안된 동작**
 - 해결: `className`, `htmlFor` 사용. ESLint로 자동 검출.
- **리스트 렌더링 시 `key` 누락 경고**
 - 원인: `map()` 으로 생성한 엘리먼트에 `key` 없음.
 - 해결: `key={id}` 같은 고유 키 추가. 인덱스는 특별한 경우 제외 권장.
- **중괄호 내부에 `statement`(예: `if`) 사용 시 오류**
 - 해결: 조건은 삼항/논리 연산자로 처리하거나, JSX 바깥에서 로직 처리.

- **컴포넌트 이름이 소문자여서 커스텀 컴포넌트로 인식되지 않음**
 - 규칙: 컴포넌트는 대문자로 시작 (`MyButton`), 소문자는 HTML 태그로 처리됨.
- **0 && <A/> 때문에 0이 출력되는 문제**
 - 해결: `condition ? <A/> : null` 사용하거나 조건값을 Boolean으로 변환.
- **XSS 위험: `dangerouslySetInnerHTML` 사용 주의**
 - 안전하지 않은 HTML을 그대로 삽입하면 XSS 취약. 가능한 파싱·정화(library 사용) 권장.
- **이벤트 핸들러에서 `this` 문제(클래스 컴포넌트)**
 - 해결: 핸들러 바인딩(생성자에서 `.bind(this)`), 또는 화살표 함수 사용, 함수형 컴포넌트 + `hook` 권장.
- **속성 스프레드 순서 문제**
 - 예: `<Comp {...base} prop="override" />` — 명시적 prop이 뒤에 오면 덮어씀. 순서 유의.

1.2 React DOM 렌더링 구조

React는 Virtual DOM을 통해 최소한의 변경만 실제 DOM에 반영함으로써 성능을 최적화 하며, ReactDOM의 역할과 렌더링 과정, 그리고 React 18에서 19로 이어지는 변화까지 이해하면 효율적인 UI 갱신 구조를 명확히 파악할 수 있다.

(1) Virtual DOM 개념과 실제 DOM과의 차이

Virtual DOM은 메모리 상에서 UI 구조를 가볍게 표현한 트리, 직접 DOM 조작 대신 변경된 부분만 계산하여 실제 DOM에 반영함으로써 성능을 개선한다.

Virtual DOM(이하 VDOM)은 실제 DOM의 경량화된 자바스크립트 표현(트리)으로, 상태 변경이 발생하면 전체 UI를 직접 조작하는 대신 VDOM을 새로 만들고 이전 VDOM과 비교하여(=reconciliation) 최소한의 실제 DOM 변경만 적용한다는 아이디어입니다. VDOM을 여러 번 재생성해 비교하는 작업은 메모리·CPU 상에서 훨씬 빠르게 처리되고(자바스크립트 객체 조작이 브라우저 DOM 조작보다 비용이 낮음), 그 결과로 실제 DOM 조작 횟수와 레이아웃/페인트 비용을 줄여 사용자 체감 성능을 향상시킵니다. 이 과정(VDOM 생성 → 비교 → 최소 변경 적용)을 **reconciliation**이라고 부릅니다.

(2) ReactDOM.createRoot()와 render() 방식 비교

React 18부터는 createRoot()를 사용해 병렬적 렌더링을 지원하고, 이전 render() 방식보다 효율적인 업데이트와 동시성을 제공한다.

- `ReactDOM.render()` 는 레거시 엔트리 방식(React 17 이전 스타일). React 18에서 deprecated 되었고, React 19에서는 제거됩니다. 대신 `createRoot()` (`react-dom/client`)로 루트를 만들고 `root.render()` 를 호출하는 방식이 표준이 되었습니다. 이는 동시성(concurrency) 기능, 자동 배칭(automatic batching), 새로운 스케줄러 동작 등 현대적 렌더링 엔진을 제공하기 위한 진입점입니다.

예제 (권장: createRoot)

```
// index.js (React 18/19 권장)
import { createRoot } from 'react-dom/client';
import { StrictMode } from 'react';
import App from './App';

const domNode = document.getElementById('root');
const root = createRoot(domNode); // new root API
root.render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

왜 바뀌었나 (기술적 배경)

- `createRoot()` 는 React의 **Fiber 스케줄러**와 결합되어 **업데이트 우선순위 관리(스케줄링)**, 작업 중단/재개(incremental rendering), 사용자 입력 우선 처리 같은 동시성 기능을 활용합니다. 또한 React 18부터 도입된 **자동 배칭**은 `createRoot()` 가 사용될 때 전역적으로 적용되어 여러 상태 업데이트를 하나의 렌더로 묶어 불필요한 재렌더를 줄입니다.
[ReactGitHub](#)

React 19에서의 추가/제거

- React 19에서는 `ReactDOM.render` 등 레거시 API가 완전히 제거됩니다(마이그레이션 가이드 존재). `root.unmount()` / `hydrateRoot()` 등 새로운 API 사용을 권장합니다. 업그레이드 가이드와 릴리스 노트를 통해 변경점과 codemod를 제공하므로 대규모 코드베이스는 단계적 마이그레이션이 필요합니다.

(3) DOM 업데이트 최소화 전략

React는 두 Virtual DOM을 비교하여 바뀐 부분만 실제 DOM에 반영하는 Diffing 알고리즘을 사용해 불필요한 렌더링을 방지한다.

① Diffing의 핵심 아이디어

- React는 두 VDOM 트리를 루트에서부터 재귀/트리 방식으로 비교합니다. 같은 타입(type)이면 속성만 갱신하려 시도하고(재사용). 타입이 바뀌면 해당 서브트리를 완전히 언마운트하고 새로 마운트합니다. 이 단순한 규칙 덕분에 전체 트리를 최적화된 방식으로 $O(n)$ 수준으로 비교할 수 있습니다(완전한 트리-대-트리 최적 비교는 비용이 높기 때문).

② keys의 역할

- 형제 리스트(예: `items.map(...)`)에서 React는 `key` 를 사용해 각 항목의 정체성(identity)을 추적합니다. 적절한 고유 키를 주면 항목의 재사용·이동·삭제를 정확히 처리할 수 있어 DOM 재생성을 최소화합니다. 인덱스를 key로 쓰면 재정렬/삽입/삭제 시 잘못된 재사용으로 DOM·상태 불일치가 발생할 수 있습니다. legacy.reactjs.org

③ 실제 적용 규칙

1. 요소 타입이 동일하면 속성/자식 비교를 계속한다.
2. 타입이 다르면 기존 노드 전부 제거 후 새로 생성(상태 손실 주의).
3. 리스트는 key 기반 매칭을 사용해 최소 변경만 수행.
4. 컴포넌트 내부에서 불필요하게 새 객체/함수를 props로 넘기면 매번 다른 prop으로 인식되어 재렌더 유발 — `useMemo` , `useCallback` 고려.

(4) UI 갱신 흐름

State가 변경되면 Virtual DOM이 다시 생성되고 Diffing 과정을 거쳐 실제 DOM이 갱신되며, 이 흐름을 이해하면 React의 선언적 렌더링 원리를 명확히 알 수 있다.

- 상태 변경(예: `setState` , `dispatch`)이 트리거된다.
- React는 해당 컴포넌트(및 필요 시 하위 컴포넌트)를 재렌더(함수 호출) 하여 새 VDOM을 생성한다. (render 단계) React
- 새 VDOM과 이전 VDOM을 비교(diff/reconciliation) 해서 변경된 최소한의 작업을 식별한다.

- 커밋 단계에서 실제 DOM에 변경을 적용(속성 업데이트, 노드 추가/삭제)하고, 라이프 사이클 훅/효과(`useEffect` 의 `cleanup` 및 실행)는 커밋 시점에 맞춰 수행된다.
- React의 스케줄러(Fiber)가 우선순위를 정해 렌더 작업을 분할/일시중단/재개할 수 있으며, 사용자 입력(고우선)과 비핵심 업데이트(저우선)를 구분해 UX를 부드럽게 유지한다.

(5) ReactDOM의 역할과 index.js 파일 구조

index.js 파일에서 ReactDOM은 루트 컴포넌트를 지정하고 전체 UI 트리를 관리하며, 이벤트 연결과 UI 업데이트의 시작점을 담당한다.

① ReactDOM의 역할

- 애플리케이션의 **루트 DOM 노드**를 점유하고, React 렌더러(Fiber)와 브라우저 DOM 사이의 연결을 관리합니다. 루트 생성(`createRoot`) → 렌더(`root.render`) → 언마운트(`root.unmount`) 같은 라이프사이클 API를 제공하며, 하이드레이션(`hydrateRoot`)·리소스 프리로딩 등 SSR 연동을 지원합니다.

② 권장 index.js 구조 (Vite / CRA 공통 권장)

```
import { createRoot } from 'react-dom/client';
import { StrictMode } from 'react';
import App from './App';
import './index.css';

const container = document.getElementById('root');
const root = createRoot(container);

root.render(
  <StrictMode>
    <App />
  </StrictMode>
);

// 개발 중 HMR/fast refresh는 도구(예: Vite)가 자동으로 처리
```

- **StrictMode**로 감싸면 개발 환경에서 추가 검사(잠재 버그 감지)가 활성화됩니다.

1.3 단일 루트 노드 개념

React 컴포넌트는 반드시 하나의 루트 노드를 반환해야 하며, 여러 형제 요소를 감싸는 방법과 Fragment 활용 방식을 익히면 불필요한 DOM 생성을 줄이고 코드 가독성을 높일 수 있다.

(1) 단일 루트 노드가 필요한 이유

렌더링 트리를 일관성 있게 유지하기 위해 컴포넌트는 반드시 하나의 루트 노드를 반환해야 하며, 그렇지 않으면 오류가 발생한다.

- **일관된 렌더 트리:** 컴포넌트가 하나의 루트(React 엘리먼트)를 반환하면 React는 그 루트를 트리의 한 노드로 취급하여 하위 트리를 안정적으로 관리·교체·갱신할 수 있습니다. 여러 루트를 허용하면 어느 부분이 컴포넌트 경계인지 불명확해져 reconciliation이 복잡해집니다.
- **JSX 문법 제약:** JSX 표현식은 하나의 루트 엘리먼트로 감싸져야 올바른 문법으로 파싱됩니다(`Adjacent JSX elements must be wrapped in an enclosing tag` 오류).
- **DOM 위치·구조 보장:** 단일 루트는 컴포넌트를 DOM 트리에 안전하게 삽입/제거할 수 있게 해 줍니다(특히 마운트/언마운트 시 상태·이펙트 정리에 유리).

(2) 여러 형제 요소를 반환할 때 발생하는 오류

여러 태그를 감싸지 않고 반환할 경우 “Adjacent JSX elements must be wrapped” 오류가 발생하며, 이를 방지하기 위해 상위 태그가 필요하다.

- **일반 오류 메시지 (개발자 경험):**

`Adjacent JSX elements must be wrapped in an enclosing tag` (또는 "컴포넌트가 여러 루트 엘리먼트를 반환하려고 함")

- **원인:** 컴포넌트의 `return` 부분에서 `<h1/><p/>` 처럼 여러 요소가 바로 나열되어 있을 때 발생.
- **해결:** 상위 태그로 감싸거나 Fragment 사용.

```
// ❌ 오류 발생
function Bad() {
  return (
    <h1>Title</h1>
    <p>Paragraph</p>
  );
}

// ✅ 해결 1: div로 감싸기
function GoodDiv() {
  return (
    <div>
      <h1>Title</h1>
      <p>Paragraph</p>
    </div>
  );
}

// ✅ 해결 2: Fragment 사용 (DOM에 추가 노드 없음)
function GoodFrag() {
  return (
    <><h1>Title</h1>
      <p>Paragraph</p>
    </>
  );
}
```

(3) <div> vs <React.Fragment> vs <> </> 활용 방법

형제 요소를 감쌀 때 불필요한 DOM 요소를 만들고 싶지 않다면 Fragment나 축약 문법을 사용하고, 필요한 경우에는 <div>를 활용한다.

- <div>
 - 장점: DOM에 실제 요소가 있으므로 CSS 그리드/플렉스 레이아웃에서 컨테이너로 활용 가능. 클래스나 스타일을 줄 수 있음.

- 단점: 불필요한 DOM 노드가 늘어나 레이아웃/접근성/성능 문제를 일으킬 수 있음 (`div soup`).
- `<React.Fragment>`
 - 장점: DOM에 아무 노드도 생성하지 않음 — siblings를 그룹화하면서 DOM을 깔끔하게 유지. `key` 속성을 줄 수 있음(중요!).
 - 단점: `className`, `style` 등 속성 불가.
- `<>...</>` (축약형)
 - 장점: 문법이 더 간결함.
 - 단점: `key` 같은 속성을 줄 수 없음(속성 필요 없을 때 사용).

언제 무엇을 쓸지

- 레이아웃 제어(컨테이너 필요)나 스타일/ARIA 속성을 줘야 하면 `<div>` 또는 시맨틱 태그 사용.
- 단지 여러 자식을 그룹화하고 DOM을 늘리고 싶지 않을 때는 `<>...</>` 또는 `React.Fragment`.
- mapping에서 그룹에 `key` 가 필요하다면 `React.Fragment key={...}` 사용(축약형은 불가).

```
// key가 필요한 경우(축약형 사용 불가)
items.map(item => (
  <React.Fragment key={item.id}>
    <dt>{item.term}</dt>
    <dd>{item.definition}</dd>
  </React.Fragment>
));
```

(4) Fragment 사용의 장점

Fragment는 DOM에 불필요한 노드를 추가하지 않으면서 여러 자식을 그룹화할 수 있어 성능과 구조적 단순성을 동시에 확보할 수 있다.

- **DOM 경량화:** 불필요한 래퍼 엘리먼트를 생성하지 않으므로 레이아웃 재계산(reflow)과 페인트 비용을 줄여 성능 유리.
- **유효한 시맨틱 HTML 유지:** 예컨대 `<ul>` 내부에 불필요한 `<div>` 를 넣으면 `<li>` 구조가 깨질 수 있는데, Fragment는 그런 문제를 방지.

- **접근성(A11y) 및 CSS 영향 최소화:** 스크린리더가 DOM 구조를 읽을 때 불필요한 노드가 없으므로 의미 흐름이 줄어들음.
- **키 사용으로 그룹 식별 가능:** 그룹화된 자식 묶음에 key를 달아 리스트 재정렬 시 React가 올바르게 처리하도록 할 수 있음(`React.Fragment key="..."`).

예: 테이블 행 렌더링(잘못된 예/좋은 예)

```
// 잘못된 예: <div>로 감싸면 트리 구조가 깨짐(테이블 내부 규칙 위반)
<tbody>
  {data.map(r => (
    <div key={r.id}>
      <tr><td>{r.a}</td></tr>
      <tr><td>{r.b}</td></tr>
    </div>
  ))}
</tbody>

// 좋은 예: Fragment 사용 (DOM에 extra node 없음)
<tbody>
  {data.map(r => (
    <React.Fragment key={r.id}>
      <tr><td>{r.a}</td></tr>
      <tr><td>{r.b}</td></tr>
    </React.Fragment>
  ))}
</tbody>
```

(5) 컴포넌트 반환 규칙과 코드 예제

컴포넌트는 하나의 루트 노드를 반환해야 하며, Fragment나 `<div>` 로 감싸는 방식을 코드 예제를 통해 명확히 이해할 수 있다.

① 반환 가능한 타입

- **React 엘리먼트** (요소 하나가 루트)
- **문자열, 숫자** (텍스트 노드로 렌더링)

- `null` 또는 `false` (렌더링하지 않음)
- 배열(Array) of elements (각 요소에 `key` 필요)
- Portals (`createPortal` 로 다른 DOM 노드에 렌더링)
- Fragments (`React.Fragment` / `<>`)

② 함수형 컴포넌트 예제

```
// 단일 루트(권장)
function Card() {
  return (
    <article className="card">
      <h2>Title</h2>
      <p>Content</p>
    </article>
  );
}

// null 반환 (조건부 렌더링)
function Maybe({ show }) {
  if (!show) return null;
  return <div>Visible</div>;
}

// 배열 반환 (key 필수)
function List() {
  return [
    <li key="a">A</li>,
    <li key="b">B</li>
  ];
}
```

③ 클래스 컴포넌트 (render 규칙 동일)

```
class MyComp extends React.Component {
  render() {
```

```

return (
  <><h1>Hi</h1>
    <p>Welcome</p>
  </>
);
}
}

```

④ 고급: map에서 fragment key 사용 예

```

const items = [
  { id: 1, term: 'CPU', def: 'Central Processing Unit' },
  { id: 2, term: 'RAM', def: 'Random Access Memory' }
];

function Glossary() {
  return (
    <dl>
      {items.map(item => (
        <React.Fragment key={item.id}>
          <dt>{item.term}</dt>
          <dd>{item.def}</dd>
        </React.Fragment>
      ))}
    </dl>
  );
}

```

1.4 개발환경 구성 및 프로젝트 생성

React 개발을 시작하려면 Node.js와 패키지 매니저 설치를 확인하고, 최신 프로젝트 생성 도구(Vite 등)를 활용하여 환경을 구성하며, 기본 구조와 실행 과정을 익히면 이후 컴포넌트 개발의 토대가 된다.

(1) Node.js와 npm/yarn 설치 확인

React 실행을 위해 Node.js 런타임과 패키지 매니저(npm 또는 yarn)가 필수이며, 버전 확인과 설치 과정을 통해 개발 환경을 준비한다.

- 설치 확인

```
node -v
npm -v    # 또는 yarn -v / pnpm -v
```

- 권장 정책
 - 빌드 도구(특히 최신 Vite)는 최신 Node LTS 또는 그 이상을 요구합니다. (Vite는 Node 20.19+ / 22.12+ 등 최신 범위를 요구합니다 — 로컬 환경은 공식 문서 권장 버전 사용). [vitejs](#)
- 버전 관리
 - 여러 프로젝트에서 버전이 달라질 수 있으니 `nvm` / `n` / `volta` 등으로 Node를 관리하세요. (예: `nvm install 20 && nvm use 20`)
- 패키지 매니저 선택
 - npm(보편적), pnpm(빠른 설치·효율적 캐시), yarn(팀 선호) 중 선택. CI 환경과 일관되게 사용하세요.

실무 팁: 새 프로젝트는 최신 LTS/권장 Node로 생성한 뒤, CI에서 동일 버전을 고정(예: `.nvmrc` 또는 `engines` 필드)하세요.

(2) 최신 React 19 프로젝트 생성 방법

CRA, Vite, Next.js 등 다양한 프로젝트 생성 도구가 있으며, 최신 React 19에서는 빠른 빌드와 개발 환경 제공 측면에서 Vite가 주로 활용된다.

- **CRA (Create React App):** 전통적이고 쉬우나 빌드 속도/개발 경험이 Vite 대비 느림. 복잡한 커스터마이징 시 eject 필요(번거로움).
- **Vite:** 빠른 HMR, 가벼운 번들링, 템플릿 생성을 통한 빠른 시작이 가능해 현재 권장되는 선택지입니다(React + Vite 조합이 표준화됨). [Reactvitejs](#)
- **Next.js:** SSR, SSG, RSC(React Server Components) 등 생산급 기능이 필요한 경우 선택 (단순한 SPA엔 과한 경우도 있음).

권장: 교육/강의/실습 목적이거나 빠른 dev 경험을 원하면 **Vite 기반**(또는 Vite + TypeScript)으로 시작하세요.

(3) Vite 기반 React 프로젝트 생성 실습

`npm create vite@latest my-app` 명령어를 통해 React 프로젝트를 손쉽게 생성할 수 있으며, 설정이 간단하고 빠른 개발 서버를 제공한다.

- 기본 생성 (터미널에서)

```
# 가장 단순한 방법 — 프롬프트에 따라 템플릿 선택 가능
npm create vite@latest my-app
# 또는 템플릿을 바로 지정
npm create vite@latest my-app -- --template react
# TypeScript 템플릿
npm create vite@latest my-app -- --template react-ts
```

- Windows PowerShell 주의
 - Windows PowerShell에서는 `--template` 인자 전달 문제가 있어, triple-dash(`- - -`)를 써야 하는 경우가 있습니다. (cmd / PowerShell 환경 차이로 인한 팁) — 필요하다면 `npx create-vite@latest my-app - - - --template react` 같은 방식 사용.
- 설치 및 실행

```
cd my-app
npm install # 또는 pnpm install / yarn
npm run dev # 개발 서버 실행
```

- package.json의 주요 스크립트

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview"
}
```

실무 팁: TypeScript 프로젝트는 `--template react-ts` 로 시작하면 타입 설정과 `tsconfig.json` 이 자동으로 구성되어 편합니다.

(4) 필수 개발 도구 설치

VS Code, ESLint, Prettier, React DevTools와 같은 도구를 설치하면 코드 품질을 유지하고 디버깅 및 협업 효율성을 높일 수 있다.

- 에디터: **VS Code** 추천 — 확장(Extensions)으로 생산성 확보.
- 필수 VS Code 확장
 - ESLint, Prettier, EditorConfig, Tailwind CSS IntelliSense(사용시), GitLens 등.
- ESLint + Prettier 설정(예시)
 - 설치

```
npm install -D eslint eslint-plugin-react eslint-plugin-react-hooks \
eslint-config-prettier prettier
```

- `.eslintrc.cjs`

```
module.exports = {
  root: true,
  env: { browser: true, es2024: true },
  extends: ['eslint:recommended', 'plugin:react/recommended', 'prettier'],
  plugins: ['react', 'react-hooks'],
  parserOptions: { ecmaFeatures: { jsx: true }, ecmaVersion: 'latest', sourceType: 'module' },
  settings: { react: { version: 'detect' } },
  rules: {
    'react/react-in-jsx-scope': 'off', // automatic runtime 사용 시
    'react/jsx-uses-react': 'off'
  }
};
```

- `.prettierrc`

```
{ "printWidth": 100, "singleQuote": true, "trailingComma": "es5" }
```

- React Developer Tools (브라우저 확장) — 컴포넌트 트리/Profiler 확인 필수.

(5) 프로젝트 구조 분석

생성된 프로젝트의 `src`, `public` 폴더와 `package.json` 파일을 분석하면 컴포넌트 작성 위치와 빌드 설정을 명확히 이해할 수 있다.

- `index.html` (프로젝트 루트): Vite 템플릿의 진입 HTML, 스크립트는 `type="module"`, 빌드 시 변환됨.
- `public/`: 정적 파일(퍼블릭 자원)을 둘 때 사용(예: favicon).
- `src/`: 개발 중 모든 소스 코드(권장 구조)

```
src/  
  main.jsx    → 루트 진입점 (createRoot 호출)  
  App.jsx     → 루트 컴포넌트  
  components/ → 재사용 컴포넌트  
  pages/      → 화면 단위 컴포넌트 (프로젝트 규모 커질 때)  
  styles/     → CSS / Tailwind 설정  
  assets/     → 이미지 등
```

- `package.json`: 의존성·스크립트·설정(예: `"type": "module"`) 확인.
- Vite의 기본 포트는 `5173` (환경에 따라 변경)이고, 개발 서버 메시지로 포트 확인 가능합니다.

예시 `main.jsx` (React 19 기준 권장 패턴)

```
import { createRoot } from 'react-dom/client';  
import App from './App';  
import './index.css';  
  
const root = createRoot(document.getElementById('root'));  
root.render(  
  <App />  
);
```

참고: React 19의 자동 JSX 런타임/변환에 따라 `import React from 'react'`가 불필요한 경우가 많습니다 — 빌드 설정(템플릿)에 따라 달라지므로 템플릿 생성 시 확인하세요.

(6) 개발 서버 실행 및 기본 페이지 확인

`npm run dev` 명령으로 개발 서버를 실행해 기본 페이지가 정상 출력되는지 확인하며, 브라우저 자동 새로고침을 통해 개발 효율을 높인다.

1. 실행

```
npm run dev
# 터미널에 뜨는 URL (기본: http://localhost:5173)로 접속
```

2. 빠른 확인 포인트

- 브라우저에서 개발자 도구(React DevTools)로 컴포넌트 트리를 확인. React
- HMR(Hot Module Replacement)이 정상 동작하는지 확인: 소스 변경 후 자동 반영 여부 체크.

3. 포트/호스트 변경 (vite.config.js)

```
// vite.config.js 예
export default {
  server: { host: true, port: 3000 } // 원하는 포트 설정
}
```

실무 팁: 개발 중 CORS/API 프록시는 `vite.config.js`의 `proxy` 옵션으로 설정해 백엔드와 로컬 연동을 간단히 처리하세요.

(7) 초기 불필요 파일 정리 및 첫 컴포넌트 작성

프로젝트 생성 시 포함된 불필요한 예제 파일을 정리하고, App 컴포넌트를 간단히 수정해 "Hello React"를 출력하며 첫 실행을 완성한다.

1. 초기 불필요 파일 정리

- 템플릿이 생성한 예제 컴포넌트/이미지/설명 파일 정리(프로젝트 목적에 맞춰 제거).
- `.gitignore` 확인(`node_modules` , `.env.local` 등 포함).

2. 폴더 구조 기본화

```
src/  
  components/  
  ui/          → 버튼, 입력 등 재사용 컴포넌트  
  pages/  
  hooks/  
  utils/
```

2. 첫 컴포넌트(예: `src/components/Button.jsx`)

```
export default function Button({ children, onClick, type='button' }) {  
  return (  
    <button type={type} onClick={onClick} className="px-4 py-2 rounded">  
      {children}  
    </button>  
  );  
}
```

3. App에 연결

```
import Button from './components/Button';  
  
export default function App() {  
  return (  
    <main>  
      <h1>Hello React 19 + Vite</h1>  
      <Button onClick={() => alert('clicked')}>클릭</Button>  
    </main>  
  );  
}
```

실무 팁: 초기부터 컴포넌트 디렉터리(재사용·UI 라이브러리), hooks 폴더, constants/utils 분리를 하면 대규모로 확장할 때 리팩토링 비용을 줄일 수 있습니다.